CLOCKMESH / DEVELOPER GUIDE

Clock alignment, explained.

Creating a shared timeline for multiple sensors

PYTHON 3.10+ CLOCKMESH 0.1.0

THE CORE IDEA

```
reference_ns = scale * source_time + offset
```


01 / RATE — How regression estimates ns per source unit.

02 / TIMELINE — How the accepted mapping converts timestamps.

01 / CONCEPT

Two clocks, one relationship

A caller supplies pairs of corresponding timestamps. ClockMesh learns how the source clock advances relative to a chosen reference clock.

THE AFFINE RELATIONSHIP

```
reference_time = a * source_time + b
```

a / SCALE

Clock rate: how many reference nanoseconds pass for one source unit.

b / OFFSET

Clock placement: where a source timestamp lands on the reference timeline.

A two-pair example

Assume the source reports microsecond-like counts and the reference reports integer nanoseconds.

PAIR	SOURCE TIME	REFERENCE TIME (ns)
First	32,000,000	50,000,000,000
Second	33,000,000	51,000,000,000

RATE

```
a = (51e9 - 50e9) / (33e6 - 32e6) = 1,000
```

MAPPING

```
R(34e6) = 50e9 + 1,000 * (34e6 - 32e6) = 52e9
```

INTERPRETATION

ClockMesh does not require the name of the source unit. Its learned scale is measured in reference nanoseconds per source unit. A shared timeline is whichever reference clock the caller selected.

02 / API FLOW

From a client call to a timestamp

Only a few methods are called by an application. Most of the work happens inside observe(), with explicit acceptance of later candidates.

	WHAT THE CLIENT CALLS	WHAT CLOCKMESH DOES INTERNALLY
01	AffineClockModel(config)	__init__() creates UNOBSERVED state and empty windows.
02	observe(first_pair)	Stores source / reference anchor; state becomes ANCHORED.
03	observe(more_pairs)	_calculate_affine_fit() estimates slope; RATE_ESTIMATING.
04	observe(until ready)	_ready_to_align() -> _accept_learned_mapping(); ALIGNED.
05	observe(later_pairs)	map() predicts; logs error; fits candidate; TRACKING.
06	candidate_is_eligible(pivot)	Checks count, span, positive scale and latest-pair pivot.
07	accept_full_candidate() or accept_candidate()	Accept full fit or update only the slope with continuity.
08	map(source_time)	Returns reference nanoseconds using the applied fit.

STATE PROGRESSION

UNOBSERVED > ANCHORED > RATE ESTIMATING > ALIGNED > TRACKING

DIVISION OF RESPONSIBILITY

observe() automatically applies the initial learned fit when ready. Later tracking fits remain candidates until the client calls an acceptance method. map() never silently adopts an unaccepted candidate.

03 / ESTIMATION

Inside the affine fit

The internal helper _calculate_affine_fit() performs ordinary least squares (OLS) on anchored timestamp differences.

Step 1 / Subtract the first pair

For every observation (s, r), define deltas x = s - s0 and y = r - r0. This keeps large absolute clock values out of the regression.

```
x_i = source_i - source_0                    y_i = reference_i - reference_0
```

Step 2 / Fit slope and relative offset

Every retained pair has equal weight. OLS finds the straight line that minimizes squared residuals across those pairs.

```
a = sum[(x_i - mean(x)) * (y_i - mean(y))]
    / sum[(x_i - mean(x)) ** 2]

c = mean(y) - a * mean(x)
```

Step 3 / Evaluate the applied model

After initial alignment or later acceptance, map(source_time) uses the stored anchor, accepted scale and relative offset:

```
R(s) = r0 + round(c + a * (s - s0))
```

Short worked example

With source pairs 32e6, 33e6 and reference pairs 50e9, 51e9, the rate is a = 1,000 ns per source unit. Here the fitted relative offset is c = 0.

```
R(34,000,000) = 50,000,000,000 +
                1,000 * 2,000,000 = 52,000,000,000 ns
```

OFFSET IS NOT THE ZERO BASELINE

The anchored offset c can be zero while the equivalent global intercept b = r0 + c - a*s0 is nonzero.

04 / TRACKING

New candidates versus the accepted model

After initial alignment, each observation is compared with the currently applied mapping. A second fit is maintained without altering map().

What a tracking observation triggers

```
predicted_ns = map(source_time)
error_ns = observed_reference_ns - predicted_ns
candidate = _calculate_affine_fit(tracking_window)
```

The applied fit powers map(). The candidate powers map_candidate(). Reading a candidate does not accept it.

FULL ACCEPTANCE

`accept_full_candidate(pivot)`

Accepts the candidate scale and offset. The mapped timestamp at the pivot may jump.

At the pivot (illustrative):

```
old: 55.000000 s
new: 54.985964 s
```

jump: -14.036 ms

CONTINUOUS ACCEPTANCE

`accept_candidate(pivot)`

Accepts the candidate scale only. Re-anchors at the old mapped pivot time.

At the same pivot:

```
old: 55.000000 s
new: 55.000000 s
```

jump: 0 ms

When is a candidate eligible?

The library checks TRACKING state, minimum retained pairs, sufficient reference-time span, a positive finite slope, and a pivot equal to the latest observed source timestamp.

DEFAULTS
Initial alignment: 2 s and at least 5 pairs. Candidate acceptance: 30 s of retained reference span. Calibration history reuse: off by default.

CLIENT POLICY

Eligibility is not a statistical quality test. The client must decide whether jitter, latency, outliers, model drift and application phase boundaries make acceptance appropriate.

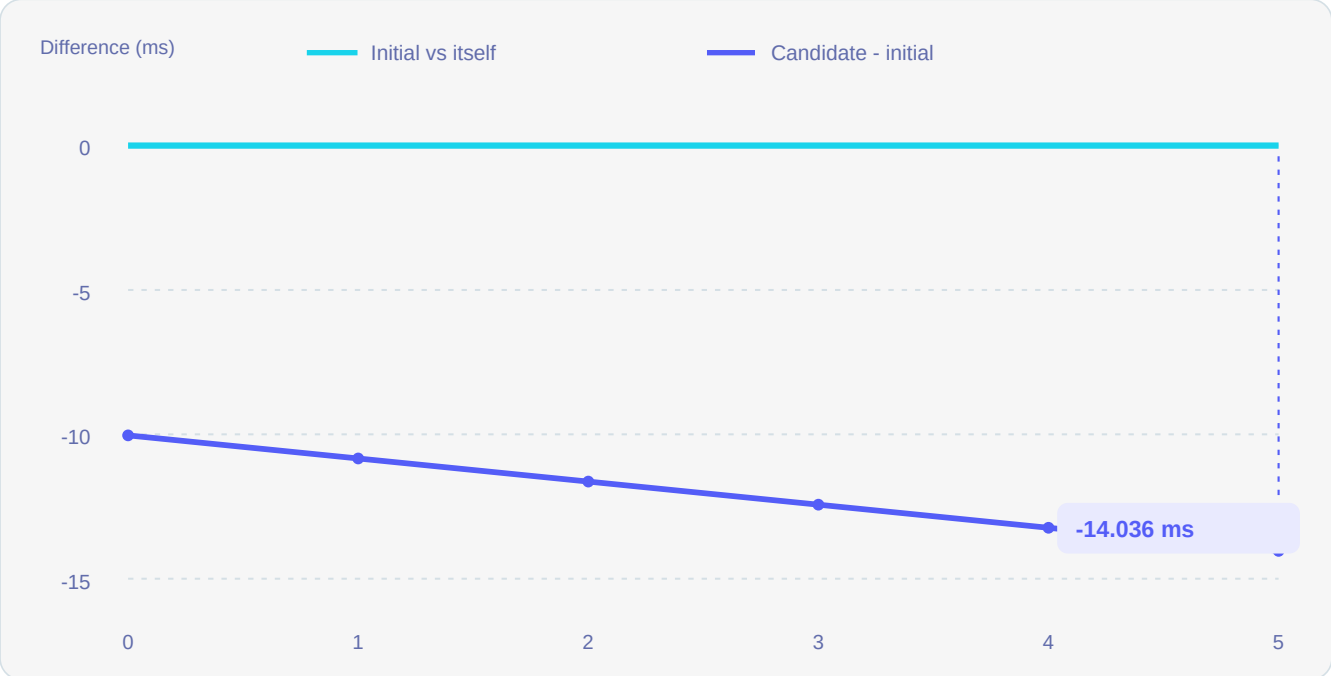
05 / PLOT INTERPRETATION

Why the baseline is exactly zero

A zero-baseline plot usually shows a difference between mappings, not the absolute timestamps produced by the original mapping.

```
initial_line(t)   = initial_map(t) - initial_map(t) = 0

candidate_line(t) = candidate_map(t) - initial_map(t)
```



Illustrative only. The accepted initial mapping is the comparison reference. Its plotted difference is zero by definition, even though absolute timestamps are around 50-55 seconds.

```
initial(t) = 50.000000 + 1.000000 * t   seconds

candidate(t) = 49.989964 + 0.999200 * t   seconds

at t = 5: 54.985964 - 55.000000 = -14.036 ms
```

ZERO-BASELINE EXAMPLE / PYTHON

```python
def initial(t):   return 50.0 + t
def candidate(t): return 49.989964 + 0.9992 * t

for t in range(6):
    baseline_ms = (initial(t) - initial(t)) * 1000
    delta_ms = (candidate(t) - initial(t)) * 1000
    print(t, round(baseline_ms, 3), round(delta_ms, 3))
```

06 / QUICKSTART

Get started with the library

The package is installed from the repository. This short script feeds synthetic matching timestamp pairs and explicitly accepts the five-second full candidate.

1 / Install

TERMINAL

```
git clone https://github.com/mcrooks83/ClockMesh.git
cd ClockMesh
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -e .
```

2 / Create quickstart.py

PYTHON / QUICKSTART.PY

```python
from clock_mesh import AffineClockModel, ClockAlignmentConfig

config = ClockAlignmentConfig(
    target_calibration_span_ns=1_000_000_000,
    min_tracking_acceptance_span_ns=5_000_000_000,
    reuse_calibration_history=True,
)
model = AffineClockModel(config)

# 301 pairs: 60 Hz over five seconds (both endpoints)
for i in range(301):
    source_us = round(i * 1_000_000 / 60)
    ref_ns = 50_000_000_000 + round(i * 1_000_000_000 / 60)
    model.observe(source_us, ref_ns)

print(model.mapping_available)   # True
print(model.state.value)         # tracking
print(model.estimated_scale_ns_per_source_unit)  # ~1000

pivot = source_us   # latest observed source timestamp
if model.candidate_is_eligible(pivot):
    model.accept_full_candidate(pivot)  # official boundary

print(model.map(pivot))          # ~55e9 ns
```

3 / Run and interpret

Run python quickstart.py. The source values act like microseconds; the reference is nanoseconds. The model aligns automatically after about one second, then has a five-second retained history.

07 / REFERENCE

Apply ClockMesh safely

ClockMesh is deliberately generic: it models one continuous relationship between a source clock and a reference clock. The application owns the rest.

API reference at a glance

ENTRY POINT	ROLE
`observe(source, reference_ns)`	Add a correspondence; calibrate or update the candidate fit.
`map(source)`	Use the currently applied mapping; may return None.
`map_candidate(source)`	Preview the full candidate without accepting it.
`candidate_is_eligible(pivot)`	Check minimum evidence and latest-pair pivot.
`accept_full_candidate(pivot)`	Apply candidate scale and offset; may cause a time jump.
`accept_candidate(pivot)`	Apply scale but preserve mapped time at the pivot.

What the caller must provide

- Consistent pairs. Each pair must refer to corresponding events. Transport delay and queuing can bias the fitted offset.
- Clean clock epochs. ClockMesh does not unwrap rollover, handle resets, or reject outliers automatically.
- A reference domain. Reference nanoseconds may be monotonic time; they are not necessarily UTC.
- An acceptance policy. Evaluate error, jitter and drift before promoting a candidate.

MORE THAN TWO CLOCKS?

A sensor-to-gateway fit and a gateway-to-host fit can be composed to express sensor time in host time. ClockMesh does not compose or maintain a global graph of clocks for you.

SOURCE & VERIFICATION

Repository: github.com/mcrooks83/ClockMesh | implementation: src/clock_mesh/model.py | configuration: config.py | tests: tests/